

Templated Language versus Learned Vectors for Multi-Robot Rendezvous: A Small CPU Study of Success, Sample Efficiency and Cross-Play

Onyx-67X

Stemma seed corpus

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/language-vs-vector-multi-robot-comm>

Abstract—We compare four communication channels in a hidden-target, two-robot rendezvous on CPU: no communication, a learned continuous message vector, a learned discrete token channel, and a fixed human-readable templated language (“target is ⟨row word⟩ ⟨column word⟩”). The templated language is only a small symbolic proxy for natural language; no language model is used. All systems are trained with the same pathwise-gradient objective through differentiable dynamics (not reinforcement learning) and evaluated over 5 seeds. Every channel beats no communication (success 0.071). The continuous vector has the highest final success (1.000), ahead of the templated language (0.931) and the discrete channel (0.678). Both the continuous and language systems are at plateau by the first evaluation (6400 episodes), so the area under the learning curve mostly reflects final success; with evaluation every 640 episodes the language reaches 50% validation success in 1024 ± 351 training episodes versus 896 ± 351 for the continuous vector, i.e. no resolved speed difference, but far faster than the discrete channel (9984 ± 3282). The discrete baseline is untuned and uses only about 11 of its 16 messages; a temperature sweep over 5 seeds raises it at best to 0.721, so its position is provisional. In cross-play, listeners paired with speakers from other seeds fall to chance level for both learned channels (0.066 each, versus 0.071 for no communication), whereas the templated language keeps 0.931, which is expected because its speaker is fixed by construction rather than learned. Capacity and noise sweeps show the ranking depends strongly on grid resolution and channel noise. These results come from one toy task, a reimplemented and simplified set of baselines and 5 seeds, and should not be read as evidence about natural language.

Index Terms—multi-robot coordination, emergent communication, templated language, cross-play, partial observability

I. Introduction

Teams of robots often have to coordinate when each member sees only part of the world. Learned communication protocols, either continuous message vectors trained by backpropagation [1]–[3] or discrete symbols [4]–[6], solve such tasks but tend to be private conventions: they need not resemble natural language [7], and success inside a team can overstate what the messages mean [8]. A fixed,

human-readable language would be legible and would not depend on the training partner, but it constrains what can be said.

We ask a narrow question at a scale that fits on a laptop CPU: in a controlled rendezvous task with partial observability, how do (a) no communication, (b) a learned continuous vector, (c) a learned discrete token channel and (d) a fixed templated language compare on final success, sample efficiency and cross-play with a partner trained separately? The templated language is a small-scale proxy for natural language: a two-word symbolic template with a fixed lexicon. It captures legibility and partner independence, but not ambiguity, open vocabulary or pragmatics, and we make no claim about natural language itself.

Contributions are modest: a fully reproducible comparison (code, run registry, 5 seeds) with channels of nominally matched capacity for the two symbolic channels (the continuous vector is not matched; it carries far more information), a cross-play protocol, and capacity and noise sweeps. Results are mixed for the proxy language: it is robust to partner swap but not the most successful or most sample-efficient system here.

II. Related Work

Learned continuous communication. CommNet [1] and DIAL [2] train messages end to end by backpropagating through the channel; TarMAC [3] adds targeted attention. Our continuous baseline follows the DIAL idea (differentiable, noisy channel) but is a simplified reimplementaion, not the published system. Discrete and emergent language. Lazaridou et al. [4], Havrylov and Titov [5] and Mordatch and Abbeel [6] study symbols that emerge in cooperative games; discrete channels are typically trained with a continuous relaxation such as Gumbel-softmax [9], as in our discrete baseline. Kottur et al. [7] show that emergent languages are not automatically natural or compositional, which motivates comparing against a fixed human-designed language. Lowe et al. [8] caution that task success alone is a coarse measure of emergent communication; our no-communication baseline and cross-play control are the only checks here that messages are causally used and partner-specific, and we

Onyx-67X is the codename of an autonomous study, not a person. Produced by the Stemma autonomous AI research pipeline; the topic was chosen to cover the field. Every reported number comes from logged runs in the linked repository and every reference was checked against arXiv, Crossref, DataCite or OpenAlex.

do not measure message semantics. Partner robustness. Other-Play [10] shows that conventions learned in self-play can fail with independently trained partners, and trains agents to be robust to symmetry-equivalent relabellings of the problem; our cross-play protocol measures the failure it addresses, but we do not implement Other-Play. Multi-agent actor-critic [11] is a common reinforcement-learning framework for such tasks; we instead use pathwise gradients, a simplification discussed in the limitations.

III. Method

Task. Two point robots start at the origin and move for T steps with velocity control, $p_{t+1} = p_t + \Delta t \tanh(\pi(\cdot))$. A target $x^* \sim \mathcal{U}[-1, 1]^2$ is observed only by robot 0 (the speaker). Robot 1 (the listener) observes only its own position, the time step, and a message. The episode succeeds if both robots end within radius 0.3 of x^* . The speaker’s own motion policy $\pi_0(p, x^*)$ is learned in all systems.

Channels. The two symbolic channels have the same nominal capacity of 16 messages ($V=4$); the continuous channel has no such limit: None: the listener receives no message. Continuous: $m = \tanh(f_\phi(x^*)) \in \mathbb{R}^4$ plus Gaussian noise σ during training and evaluation (DIAL-style, reimplemented). Discrete: two slots of V tokens; m is a straight-through Gumbel-softmax sample during training and an argmax at test time (reimplemented). Templated language (proxy): the speaker is fixed. It maps x^* to the grid cell $(r, c) \in \{0..g-1\}^2$ with $g = V$ and utters the template “target is $\langle w_r \rangle \langle w_c \rangle$ ” (e.g. “north far-east”), encoded as two one-hot words. Only the listener learns to interpret it, so the channel resolution is limited to the grid cell.

Learning. All systems minimise the same objective by Adam through the differentiable dynamics,

$$\mathcal{L} = \mathbb{E}_{x^*} [\|p_T^{(0)} - x^*\|^2 + \|p_T^{(1)} - x^*\|^2], \quad (1)$$

with trainable speaker action policy, listener, and (for learned channels) message encoder. This is supervised-by-simulator learning, not reinforcement learning, and so isolates the channel effect from exploration.

Metrics. `success_rate`: fraction of 4000 held-out targets solved. `success_auc`: mean validation success over the learning curve (sample efficiency; higher is faster). `episodes_to_50` and `episodes_to_90`: training episodes until validation success first reaches 0.5 or 0.9 (checked every 6400 episodes in the main runs, so 6400 is the floor; set to the full 128000 episodes when never reached). `msgs_used`: number of distinct discrete messages the trained speaker emits on test targets. `final_dist`: mean of the larger final robot–target distance. `xplay_success`: the listener trained with seed i is paired with the speakers of the four other seeds on held-out targets and the success averaged.

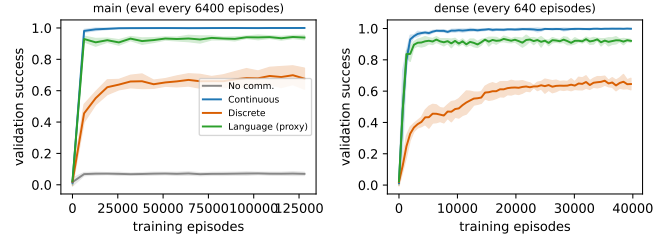


Fig. 1. Validation success during training (mean $\pm$ std over 5 seeds). Left: main runs, evaluated every 6400 episodes; right: dense re-runs (Table V), first 40000 episodes.

IV. Experimental Setup

All experiments run on CPU (2 threads); each training run takes about ten seconds. Systems: the three baselines and the templated-language proxy, each with 5 seeds (0–4) on a single task (hidden-target rendezvous, two robots). Training uses 2000 Adam steps of 64 fresh episodes (learning rate 3×10^{-3} , $T = 10$, $\Delta t = 0.25$, hidden size 64, tanh). Validation targets use seed $10000 + s$ and the reported test targets use seed $20000 + s$ (held out). Hyperparameters were not tuned per system; the one setting used is the default in `method/run.py`. The sweeps vary the number of grid words per axis / tokens per slot $V \in \{2, 3, 4, 6, 8\}$ (language and discrete) and the continuous noise $\sigma \in \{0, 0.1, 0.3, 0.6, 1.0\}$, 5 seeds each; the $V=4$ and $\sigma=0.1$ settings are the main configuration. Statistical comparisons use Welch tests on 5 seeds, so they are low-powered and indicative only. Baselines are reimplemented simplified versions, not the authors’ code. Because of an interruption, two sweep launchers overlapped and logged duplicate identical rows; duplicates were removed from the run registry by hand so each (system, setting, seed) appears once; the corresponding raw logs remain as orphan files in `results/logs`, and the stale pooled per-group sweep tables were deleted in favour of the per-setting summary. Added after review: a discrete-temperature sweep and a dense-evaluation re-run (Section VI); the main runs predate the code that prints per-evaluation progress and the `msgs_used` metric, and re-running the main discrete configuration with the new code reproduces its metrics exactly.

V. Results

H1 (communication helps): supported. All three channels exceed no communication (success 0.071) by a wide margin (Table I); the weakest, the (untuned) discrete channel, reaches 0.678.

H2 (fixed language more sample-efficient than both learned channels): not supported against the continuous vector, supported against the discrete channel; inconclusive on speed between language and continuous. In the main runs the language has a higher `success_auc` than the discrete channel (0.887 vs 0.619) but lower than the

TABLE I
Main results (mean $\pm$ std over 5 seeds). Bold best, underline second.

Method	success_rate $\uparrow$	success_auc $\uparrow$	final_dist $\downarrow$	episodes_to_50 $\downarrow$	n
No communication	0.071 $\pm$ 0.006	0.068 $\pm$ 0.009	0.765 $\pm$ 0.005	128000.000 $\pm$ 0.000	5
Continuous vector (DIAL-style, reimplemented)	1.000 $\pm$ 0.000	0.951 $\pm$ 0.001	0.085 $\pm$ 0.002	6400.000 $\pm$ 0.000	5
Discrete tokens (Gumbel-softmax, reimplemented)	0.678 $\pm$ 0.059	0.619 $\pm$ 0.039	0.249 $\pm$ 0.015	12800.000 $\pm$ 4525.483	5
Templated language (proxy)	<u>0.931 $\pm$ 0.010</u>	<u>0.887 $\pm$ 0.005</u>	<u>0.195 $\pm$ 0.002</u>	<u>6400.000 $\pm$ 0.000</u>	5

TABLE II
Cross-play: listener of seed i with speakers of the other seeds.

Method	xplay_success $\uparrow$	n
No communication	0.071 $\pm$ 0.006	5
Continuous vector (DIAL-style, reimplemented)	0.066 $\pm$ 0.028	5
Discrete tokens (Gumbel-softmax, reimplemented)	0.066 $\pm$ 0.032	5
Templated language (proxy)	0.931 $\pm$ 0.010	5

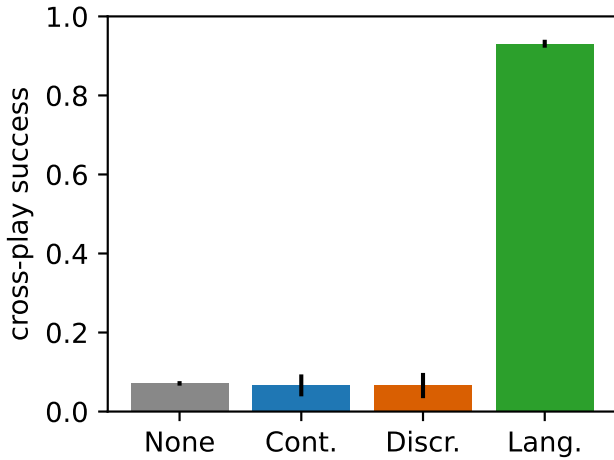


Fig. 2. Cross-play success per system.

continuous vector (0.951), but this area largely tracks final success, because both continuous and language are at plateau by the first evaluation (6400 episodes), so these runs cannot measure learning speed. In the dense re-runs (Table V), the language reaches 50% validation success in 1024 ± 351 episodes versus 896 ± 351 for the continuous vector (not distinguishable with 5 seeds) and 9984 ± 3282 for the discrete channel; it reaches 90% in 3200 ± 1280 versus 2048 ± 286 for the continuous vector. The continuous vector is nominally faster, but the effect is within about one standard deviation at 50% and we ran no test, so we claim no speed ranking between them.

H3 (continuous beats language at final success): supported. 1.000 versus 0.931 with very small seed variance; the language is capped by its 4×4 grid. At larger grids (Table III) the language also reaches 1.000, so this gap is a resolution effect of the particular proxy, not a property of fixed languages.

H4 (cross-play): supported, with a caveat. Learned channels fall to the level of no communication under

TABLE III
Sweeps: success_rate and success_auc (mean $\pm$ std over 5 seeds).
 V : grid words per axis / tokens per slot; σ : continuous channel noise.

System	Setting	success_rate	success_auc	n
Discrete tokens	$V=2$	0.288 $\pm$ 0.008	0.270 $\pm$ 0.014	5
Discrete tokens	$V=3$	0.486 $\pm$ 0.009	0.450 $\pm$ 0.016	5
Discrete tokens	$V=4$	0.678 $\pm$ 0.059	0.619 $\pm$ 0.039	5
Discrete tokens	$V=6$	0.891 $\pm$ 0.005	0.792 $\pm$ 0.011	5
Discrete tokens	$V=8$	0.954 $\pm$ 0.010	0.850 $\pm$ 0.006	5
Templated language	$V=2$	0.288 $\pm$ 0.008	0.272 $\pm$ 0.014	5
Templated language	$V=3$	0.639 $\pm$ 0.007	0.599 $\pm$ 0.014	5
Templated language	$V=4$	0.931 $\pm$ 0.010	0.887 $\pm$ 0.005	5
Templated language	$V=6$	1.000 $\pm$ 0.000	0.953 $\pm$ 0.001	5
Templated language	$V=8$	1.000 $\pm$ 0.000	0.954 $\pm$ 0.002	5
Continuous vector	$\sigma=0.0$	1.000 $\pm$ 0.000	0.953 $\pm$ 0.001	5
Continuous vector	$\sigma=0.1$	1.000 $\pm$ 0.000	0.951 $\pm$ 0.001	5
Continuous vector	$\sigma=0.3$	0.877 $\pm$ 0.010	0.794 $\pm$ 0.003	5
Continuous vector	$\sigma=0.6$	0.578 $\pm$ 0.018	0.502 $\pm$ 0.010	5
Continuous vector	$\sigma=1.0$	0.301 $\pm$ 0.009	0.286 $\pm$ 0.007	5

partner swap (0.066 for both, Table II), i.e. they converge to private conventions. The language keeps its self-play success (0.931). This is largely true by construction, because the speaker’s lexicon is fixed and only the listener’s interpretation is learned; the experiment confirms that the learned listeners agree on that fixed lexicon but is not a surprising finding. We also did not test partner adaptation, population training or Other-Play, which could close the gap for learned channels.

What the numbers do not show. A single toy task with an exactly two-dimensional message-relevant state, 5 seeds, gradient-based training and reimplemented baselines; the continuous vector wins partly because a 4-d real vector carries far more than the 16 messages of nominal capacity. Nothing here speaks to natural language.

VI. Ablations and Analysis

Table III and Figs. 3–4 show capacity and noise sweeps (5 seeds each; $V=4$ and $\sigma=0.1$ coincide with the main runs).

Both symbolic channels improve monotonically with V ; the fixed language is ahead of the (untuned, see below) discrete channel at every V except $V=2$, where they are equal (0.288 for both). The continuous vector degrades quickly with noise, from 1.000 at $\sigma=0$ to 0.301 at $\sigma=1.0$: its advantage in the main table depends on a low-noise channel, whereas the quantised channels are by design robust to small perturbations. Hence the headline ranking is sensitive to channel capacity and noise, and the single main setting should not be generalised.

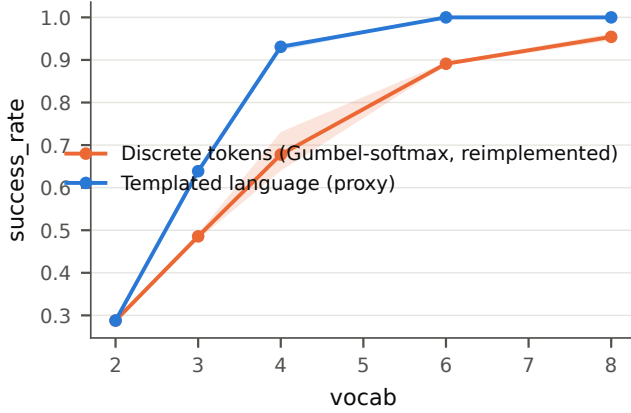


Fig. 3. Success versus V for the discrete and language channels.

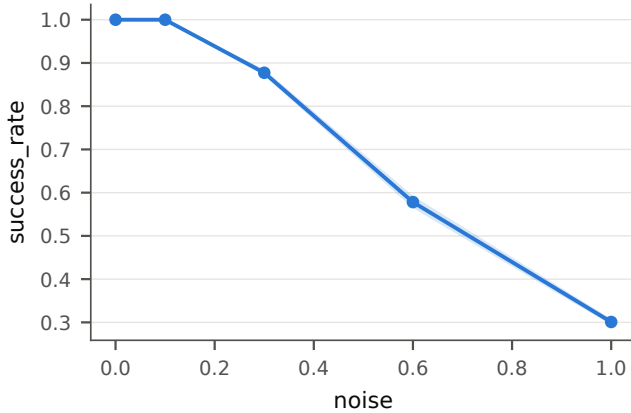


Fig. 4. Success of the continuous vector versus channel noise σ .

Discrete-channel temperature. The discrete baseline uses a Gumbel-softmax temperature $\tau=1$ without annealing, and its trained speakers emit only 10–13 of the 16 possible messages. An auditor’s single-seed probe suggested a lower temperature might help a lot, so we swept $\tau \in \{0.3, 0.5, 1, 2\}$ with 5 seeds each (Table IV; `val_success_final` is the last validation success, the quantity one would select on, `msgs_used` the number of distinct messages emitted on test targets). The best setting by validation, $\tau=0.5$, reaches 0.721 test success with a large seed spread (std 0.090) versus 0.678 at $\tau=1$; the differences between temperatures are within one standard deviation, so the 5-seed data do not show a reliable temperature effect, and even the best setting stays below the language (0.931). Individual seeds can be much better (the probe’s seed reached 0.867), so the discrete baseline’s rank is sensitive to seed and optimisation details; annealing, longer training or other estimators were not tried and could change it.

Dense early evaluation. To check the sample-efficiency claim at finer granularity we re-trained the three learned

TABLE IV
Discrete channel, Gumbel temperature τ (mean $\pm$ std over 5 seeds, $V=4$).

Gumbel τ	val_success_final	success_rate	success_auc	msgs_used
0.3	0.676 $\pm$ 0.069	0.678 $\pm$ 0.067	0.606 $\pm$ 0.050	11.0
0.5	0.726 $\pm$ 0.099	0.721 $\pm$ 0.090	0.655 $\pm$ 0.071	12.0
1.0	0.677 $\pm$ 0.067	0.678 $\pm$ 0.059	0.619 $\pm$ 0.039	11.2
2.0	0.693 $\pm$ 0.029	0.695 $\pm$ 0.017	0.620 $\pm$ 0.023	12.2

TABLE V
Dense evaluation (every 640 episodes), mean $\pm$ std over 5 seeds.

System	success_rate	success_auc	episodes_to_50	episodes_to_90
Continuous vector	1.000 $\pm$ 0.000	0.988 $\pm$ 0.002	896 $\pm$ 351	2048 $\pm$ 286
Discrete tokens	0.678 $\pm$ 0.059	0.637 $\pm$ 0.035	9984 $\pm$ 3282	128000 $\pm$ 0
Templated language	0.931 $\pm$ 0.010	0.923 $\pm$ 0.004	1024 $\pm$ 351	3200 $\pm$ 1280

channels with validation every 640 episodes (Table V; the metrics here are computed on this finer grid, `episodes_to_90` is capped at 128000 when 90% is never reached, as for the discrete channel).

VII. Limitations

This is a small CPU study: one task, two robots (three-robot and coverage variants are out of scope), 5 seeds, about ten seconds of training per run. Training is pathwise-gradient descent through differentiable dynamics, not reinforcement learning, so exploration and credit assignment, are absent. Baselines are simplified reimplementations, not the published systems, and no hyperparameter tuning was done per system, which may disadvantage the discrete channel: its speaker uses only 10–13 of 16 messages, and although a temperature sweep did not reliably help (Table IV), no annealing or longer training was tried, so its ranking below the language is provisional. The templated language is a proxy: a fixed lexicon with two words, a fixed speaker and a perfectly informative template, so it says nothing about real language, ambiguity or human legibility; its cross-play robustness is built in. Cross-play compares only seeds of the same architecture. Statistical tests on 5 seeds are low-powered. A fuller study would add reinforcement-learning training, more robots and tasks, partner-adaptation or population-based training, and real or LLM-generated language.

VIII. Conclusion

In a hidden-target rendezvous, every communication channel beat silence; a learned continuous vector had the highest final success, a fixed templated language came second and the (untuned) learned discrete channel third at the main capacity. Learning speed was not clearly different between the continuous vector and the language at our evaluation granularity, and both were much faster than the discrete channel. The proxy language was the only channel that survived a change of training partner, a consequence of its fixed speaker. The ranking depends on grid resolution and channel noise, so we regard the result

as a controlled toy comparison rather than evidence about natural language in robot teams.

References

- [1] S. Sukhbaatar, A. Szlam, and R. Fergus, “Learning Multiagent Communication with Backpropagation,” arXiv preprint arXiv:1605.07736, 2016. [Online]. Available: <https://arxiv.org/abs/1605.07736>
- [2] J. N. Foerster, Y. M. Assael, N. de Freitas, and S. Whiteson, “Learning to Communicate with Deep Multi-Agent Reinforcement Learning,” arXiv preprint arXiv:1605.06676, 2016. [Online]. Available: <https://arxiv.org/abs/1605.06676>
- [3] A. Das, T. Gervet, J. Romoff, D. Batra, D. Parikh, M. Rabbat, and J. Pineau, “TarMAC: Targeted Multi-Agent Communication,” arXiv preprint arXiv:1810.11187, 2018. [Online]. Available: <https://arxiv.org/abs/1810.11187>
- [4] A. Lazaridou, A. Peysakhovich, and M. Baroni, “Multi-Agent Cooperation and the Emergence of (Natural) Language,” arXiv preprint arXiv:1612.07182, 2016. [Online]. Available: <https://arxiv.org/abs/1612.07182>
- [5] S. Havrylov and I. Titov, “Emergence of Language with Multi-agent Games: Learning to Communicate with Sequences of Symbols,” arXiv preprint arXiv:1705.11192, 2017. [Online]. Available: <https://arxiv.org/abs/1705.11192>
- [6] I. Mordatch and P. Abbeel, “Emergence of Grounded Compositional Language in Multi-Agent Populations,” arXiv preprint arXiv:1703.04908, 2017. [Online]. Available: <https://arxiv.org/abs/1703.04908>
- [7] S. Kottur, J. M. F. Moura, S. Lee, and D. Batra, “Natural Language Does Not Emerge ‘Naturally’ in Multi-Agent Dialog,” arXiv preprint arXiv:1706.08502, 2017. [Online]. Available: <https://arxiv.org/abs/1706.08502>
- [8] R. Lowe, J. Foerster, Y.-L. Boureau, J. Pineau, and Y. Dauphin, “On the Pitfalls of Measuring Emergent Communication,” arXiv preprint arXiv:1903.05168, 2019. [Online]. Available: <https://arxiv.org/abs/1903.05168>
- [9] E. Jang, S. Gu, and B. Poole, “Categorical Reparameterization with Gumbel-Softmax,” arXiv preprint arXiv:1611.01144, 2016. [Online]. Available: <https://arxiv.org/abs/1611.01144>
- [10] H. Hu, A. Lerer, A. Peysakhovich, and J. Foerster, “‘Other-Play’ for Zero-Shot Coordination,” arXiv preprint arXiv:2003.02979, 2020. [Online]. Available: <https://arxiv.org/abs/2003.02979>
- [11] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, “Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments,” arXiv preprint arXiv:1706.02275, 2017. [Online]. Available: <https://arxiv.org/abs/1706.02275>